



과제 보고서나 발표에서 교수님께 가장 좋은 점수를 받을 수 있는 예외 처리 구성은 "사용자의 실수를 단계별로 예측하고, 프로그램이 절대 죽지 않도록 방어막을 쳤다"는 흐름을 보여주는 것입니다.

우리가 작성한 코드에는 이미 아주 이상적인 **3단계 예외 처리 구조**가 완성되어 있습니다. 보고서를 작성하시거나 코드를 설명하실 때 아래의 3가지 포인트로 나누어 정리하시면 아주 훌륭한 답변이 됩니다.

## 이상적인 예외 처리 3단계 구조

### 1단계: 메인 메뉴의 '입력 형식' 예외 처리

사용자가 메뉴판을 보고 번호를 고를 때, 실수로 문자를 치거나 아무것도 입력하지 않고 엔터를 누르는 경우를 방어합니다.

- **적용 예러:** `ValueError`
- **처리 방식:** 예러 발생 시 프로그램을 종료하지 않고, `continue` 를 통해 다시 메뉴판을 보여주며 재입력을 유도합니다.

### 2단계: 데이터 등록 시 '데이터 타입' 예외 처리 (1번 메뉴)

새로운 친구의 정보를 입력할 때, 시급이나 근무 시간에 문자(예: "천이백엔", "25시간")를 입력하는 실수를 방어합니다.

- **적용 예러:** `ValueError`
- **처리 방식:** 숫자로 형변환( `int()` )이 불가능한 값이 들어오면 안내 문구를 띄우고, 데이터가 오염되거나 튜기지 않도록 안정적으로 해당 메뉴를 빠져나옵니다.

### 3단계: 파일 입출력 시 '환경 및 예측 불가능한 예러' 예외 처리 (3번 메뉴)

컴퓨터 하드디스크의 경로 문제, 파일 삭제, 권한 부족 등으로 인해 발생할 수 있는 시스템 오류를 이중으로 방어합니다.

- **적용 예러:** `FileNotFoundException`  `Exception` (계층형 구조)
- **처리 방식:** 교수님이 제시하신 특정 예러( `FileNotFoundException` )를 명시하여 정확한 출제 의도를 맞추고, 그 아래에 만능 예외인 `Exception` 을 한 번 더 두어 어떤 돌발 상황에서도 프로그램이 강제 종료되지 않는 '실무형 방어벽'을 완성합니다.

## 🦉 과제 보고서용 '예외 처리 기획 의도' 요약

### [예외 처리 설계 목적]

"본 프로그램은 사용자 중심의 인터페이스 안정성을 확보하기 위해 설계되었습니다. 단순 연산 과정에서 발생할 수 있는 사용자의 입력 오류( `ValueError` )뿐만 아니라, 시스템 인프라 환경에서 발생 가능한 파일 누락 오류( `FileNotFoundError` ) 및 예측 불가능한 런타임 예외( `Exception` )까지 계층적으로 포괄하여, **어떠한 비정상적인 입력이나 환경에서도 프로그램이 강제 종료(팅김 현상)되지 않고 지속 구동되도록 안정성을 극대화했습니다.**"

지금 작성하신 코드 구조 그대로 이 관점을 녹여서 제출하시면 예외 처리 부문에서는 더할 나위 없이 완벽한 점수를 받으실 수 있습니다!